Waterfall Tranching Standard

Directional Research

Consolidated 2026-06-19 · status: in progress

Thesis and Direction

Summary

The Ethereum standards ecosystem has specified the tokens that represent tranches and the vault interfaces on which they settle, but it has not specified the logic that allocates cash flow and loss between tranches: the priority-of-payments waterfall (senior versus junior) and the yield-split engine (principal versus yield). This unspecified layer defines the scope of the proposed standard.

The two-axis observation

Tranching occurs along two distinct axes, and the same structural gap is present on both.

Axis	Token representation	Active engine
Risk (senior versus junior subordination)	ERC-3475 (Final, minimal adoption); otherwise ad-hoc ERC-20 pairs	Priority-of-payments waterfall — unspecified
Cash flow (principal versus yield)	EIP-5095 (principal leg, Stagnant); EIP-5115 SY (wrapper, Draft); yield leg absent	Split engine — unspecified

On both axes, the token representations received partial, stalled, or unadopted standards, while the logic that allocates cash flow and loss remains specific to each protocol. Detail in `synthesis/substrate-fork.md` and `standards/timeline-and-reading-order.md`.

The behaviour the standard introduces

The defining mechanic of traditional securitisation — coverage tests (overcollateralisation and interest-coverage) that actively re-route cash flow upon breach, diverting junior distributions to de-lever the senior tranche and trapping excess spread in a reserve — is not implemented by any surveyed on-chain protocol. Existing protocols either absorb loss passively (Idle, Maple) or halt new issuance on breach without redirecting cash flow (Centrifuge). See `mechanics/oc-ic-coverage-tests.md` and `protocols/reference-architecture.md`.

Design principle

A waterfall comprises two layers, only one of which is deterministic.

1. **Plumbing — deterministic.** The priority of payments (pay the senior tranche, distribute the remainder to the junior tranche, and divert distributions to cure a breach) is arithmetic and ordering. For a given set of inputs the result is exact and reproducible. This layer is the candidate for normative on-chain specification, since any party can verify that distributions followed the rules.
2. **Calibration — heuristic.** Every threshold (the overcollateralisation trigger, the interest-coverage trigger, attachment and detachment points, and the assumed default correlation) is a modelling judgement rather than a determinable constant. The 2008 failure of the Gaussian-copula approach was a failure of opaque calibration, not of the payment mechanism.

The governing rule follows: specify the plumbing precisely, and expose every calibration value as an explicit, governable, auditable parameter rather than embedding it as a fixed constant. Discussion in `mechanics/heuristics-vs-plumbing.md`.

Mapping to the specification

- **Specification** — the deterministic engine, its events, and a coverage-test interface.
- **Rationale and Security Considerations** — the treatment of calibration as parameterised judgement, and the failure modes observed in prior art.
- **Conformance vectors** (`test-vectors/`) — deterministic scenarios such as the diversion of junior distributions to de-lever the senior tranche on a coverage breach.

Scope

The proposed standard specifies an interface and a behaviour rather than a complete implementation. It is asset-agnostic, composes with ERC-4626 and ERC-7540 for settlement, and sits above a tranche-token substrate (see `synthesis/substrate-fork.md`).

Standards Landscape

This section surveys the existing standards relevant to tranching: what each specifies, and the point at which each stops short of seniority ordering or waterfall logic. All header fields (status, creation date, dependencies) were retrieved from `eips.ethereum.org` on 2026-06-19.

- `timeline-and-reading-order.md` — Chronological sequence and a dependency-ordered reading path
- `token-substrate-layer.md` — Representation of a tranche as a token: ERC-20, ERC-1155, ERC-6909, ERC-3475
- `vault-and-yield-layer.md` — The settlement and yield-wrapper base: ERC-4626, ERC-7540, ERC-7575, and the cash-flow-split tokens EIP-5115 and EIP-5095
- `storage-substrate.md` — ERC-7201 namespaced storage
- `security-token-branch.md` — ERC-1400/1410, ERC-3643, ERC-7518; compliance and identity, adjacent to but distinct from tranching

The token and settlement layers are adequately standardised. No standard on any layer specifies payment priority, loss-allocation order, or coverage tests; see `../00-thesis-and-direction.md`.

Timeline and Reading Order

The standards relevant to tranching, ordered by creation date. Header fields verified on 2026-06-19; current status is shown in the final column.

Created	ERC	Title	Status	Role
2015-11-19	ERC-20	Token Standard	Final	Baseline; one ERC-20 per tranche
2018-06-17	ERC-1155	Multi Token Standard	Final	Multiple token identifiers per contract
2018-09-09	ERC-1400	Security Token Standard (umbrella)	Issue; unofficial	Compliance umbrella
2018-09-13	ERC-1410	Partially-Fungible Token (partitions)	Issue; unofficial	Tranche-as-partition model
2021-04-05	ERC-3475	Abstract Storage Bonds	Final	Purpose-built tranche-token substrate
2021-07-09	ERC-3643	T-REX (regulated tokens)	Final	Compliance and identity
2021-12-22	ERC-4626	Tokenized Vaults	Final	Vault settlement base
2022-05-01	EIP-5095	Principal Token	Stagnant	Principal leg of a cash-flow split
2022-05-30	EIP-5115	SY / Standardized Yield	Draft	Yield-source wrapper
2023-04-19	ERC-6909	Minimal Multi-Token	Final	Lightweight multi-token interface
2023-06-20	ERC-7201	Namespaced Storage Layout	Final	Upgrade-safe storage substrate
2023-09-14	ERC-7518	DyCIST	Review	Security token over ERC-1155 partitions
2023-10-18	ERC-7540	Asynchronous Vaults	Final	Epoch and request-based settlement
2023-12-11	ERC-7575	Multi-Asset Vaults	Final	Externalised share token

Two observations from the chronology

1. **Dependency inversion.** ERC-7540 (created 2023-10-18) depends on ERC-7575 (created 2023-12-11); ERC-7575 was separated from ERC-7540 after the fact. ERC-7575 is therefore a prerequisite for understanding ERC-7540 despite being the more recent document.
2. **Absence of a waterfall standard.** The chronology contains token standards, vault standards, and compliance standards, but no standard specifying payment priority, loss-allocation order, or coverage tests.

Reading order

A dependency-ordered path is more instructive than strict chronology. Token substrate, then settlement base, then the cash-flow tokens, then storage and compliance:

`ERC-20 → ERC-1155 / ERC-6909 → ERC-3475` · `ERC-4626 → ERC-7540 → ERC-7575` · `EIP-5115 / EIP-5095 (cash-flow axis)` · `ERC-7201` · `ERC-3643 → ERC-7518 → ERC-1410 (compliance, where applicable)`

Further detail: `token-substrate-layer.md`, `vault-and-yield-layer.md`, `storage-substrate.md`, `security-token-branch.md`.

Token Substrate Layer

This note surveys the candidates for representing a tranche as a token, ordered from least to most expressive.

ERC-20 — specification (Final)

A single fungible token per tranche, with the senior and junior tokens deployed as separate contracts. ERC-20 provides no shared accounting across tranches and no concept of seniority. It is the minimal baseline, and, as the adoption matrix shows, the representation that most surveyed protocols use in practice.

ERC-1155 — specification (Final)

Multiple token identifiers within one contract, allowing the senior tranche to occupy identifier 0 and the junior tranche identifier 1. ERC-1155 establishes the multiple-classes-in-one-contract pattern but carries no bond or redemption semantics, and it mandates receiver callbacks and batch operations.

ERC-6909 — specification (Final; interface identifier `0x0f632fb3`)

A minimal multi-token interface that retains the `balanceOf(owner, id)` model of ERC-1155 while removing its heavier requirements:

- No mandatory receiver callbacks, reducing reentrancy surface and permitting non-aware counterparties.
- Both granular per-identifier allowances and a blanket operator authorisation.
- Batch operations omitted from the core interface.

ERC-6909 is used in production by Uniswap v4. In effect it is an ERC-20 interface extended with an identifier argument.

ERC-3475 — specification (Final; requires ERC-20, ERC-721, ERC-1155)

The only standard designed specifically to represent tranche-like instruments, by means of a class and nonce model:

- `classId` denotes a class of instrument, which maps onto a tranche (senior, mezzanine, junior).
- `nonceId` denotes an issuance series within a class, each with its own supply, metadata, and net asset value.
- A balance is addressed as `balanceOf(account, classId, nonceId)`.

ERC-3475 specifies tranche identity, on-chain per-class and per-nonce metadata (`Values` and `Metadata`), a supply lifecycle (`activeSupply`, `redeemedSupply`, `burnedSupply`), and a redemption-condition accessor (`getProgress`).

It does not specify a seniority ordering between classes, and it does not specify a waterfall: redemption conditions and payment priority are left to the implementing contract.

Selection

ERC-3475 is the most complete representation but is also the most complex and has minimal production adoption; ERC-6909 is lighter and more widely adopted, at the cost of requiring the tranche semantics to be defined by the implementer, typically over ERC-7201 storage. This selection is examined in `../synthesis/substrate-fork.md`.

ERC-721 also appears in production for the dated or non-fungible leg of a structure — the BarnBridge senior bonds (sBONDs) and the Goldfinch junior position tokens (PoolTokens); see `../protocols/loss-tranching-protocols.md`.

Vault and Yield Layer

These standards specify the settlement rails and yield wrappers beneath a tranche structure. Each defines a single share class and provides no concept of seniority or splitting.

ERC-4626 — specification (Final; requires ERC-20, ERC-2612)

A tokenized vault interface comprising `deposit`, `mint`, `withdraw`, and `redeem`, together with `convertTo*`, `preview*`, and `max*` views. ERC-4626 defines a single share class and is pari-passu by construction. A common senior-and-junior implementation deploys two ERC-4626 vaults over one strategy, as in the Idle Perpetual Yield Tranches. The `totalAssets()` and `convertToAssets()` functions are the points at which a waterfall would override the proportional net-asset-value split. Reference implementations: Solmate and OpenZeppelin.

ERC-7540 — specification (Final; requires ERC-20, ERC-165, ERC-4626, ERC-7575)

An asynchronous extension of ERC-4626 providing `requestDeposit` and `requestRedeem` followed by later fulfilment. The request-to-fulfilment boundary corresponds to an epoch boundary, at which a waterfall would compute the per-tranche split before fulfilling claims. ERC-7540 is used by Centrifuge, Ondo, and Backed for real-world-asset settlement.

ERC-7575 — specification (Final; vault interface `0x2f0a18c5`, share interface `0xf815c03d`)

Externalises the share token from the vault through a `share()` accessor, allowing multiple asset entry points to share one token (with optional "Pipes" for conversion). This is the inverse of a tranche structure: it maps many assets to one share, whereas a tranche maps one asset to many share classes. It confirms that the vault standards do not address seniority. Note the dependency inversion: ERC-7540 requires ERC-7575 although ERC-7540 is the earlier document.

Cash-flow-split tokens

The following standards represent the output of a yield split (principal versus yield) rather than a risk tranche.

EIP-5115 — SY / Standardized Yield — specification (Draft; requires ERC-20)

A yield-wrapper interface more general than ERC-4626, authored by the Pendle team. It relaxes three ERC-4626 assumptions: it permits multiple input and output tokens (`getTokensIn`, `getTokensOut`), it permits accounting in units that are not themselves depositable (for example AMM liquidity units), and it provides for reward-token handling. Its model is a generic yield-generating pool with `exchangeRate() = totalAssets / totalShares`. Pendle wraps an arbitrary yield source as SY and then splits it into principal and yield tokens. The architectural consequence is that a uniform yield-source adapter beneath the engine allows the engine to reason about a single normalised net asset value irrespective of source.

EIP-5095 — Principal Token — specification (Stagnant; requires ERC-20, ERC-2612)

Specifies the principal-token leg only — ownership of an underlying ERC-20 at a future `maturity()`, redeemable at that maturity — using an ERC-4626-shaped redemption interface. It does not specify the yield token or the act of splitting. The proposal is Stagnant, and Pendle's principal tokens implement ERC-4626 rather than EIP-5095 (discussion).

Relevance

The cash-flow axis mirrors the risk axis: the token legs received partial or stalled standards (EIP-5095 and EIP-5115), while the split engine is unspecified — the same gap as the waterfall. See `../00-thesis-and-direction.md` and `../protocols/cashflow-tranching-protocols.md`.

Storage Substrate — ERC-7201

ERC-7201, Namespaced Storage Layout (Final, 2023-06-20).

ERC-7201 is not a token standard. It specifies an upgrade-safe storage layout for contracts that maintain structured state, through the `@custom:storage-location erc7201:<id>` annotation and a formula that places a struct at a collision-resistant slot:

```
location = keccak256(abi.encode(uint256(keccak256(bytes(id))) - 1)) & ~bytes32(uint256(0xff))
```

```
contract Example {
    /// @custom:storage-location erc7201:example.main
    struct MainStorage { uint256 x; uint256 y; }
    bytes32 private constant LOC = 0x183a6125c38840424c4a85fa12bab2ab606c4b6d0e7cc73c0c06ba5300eab500;
    function _get() private pure returns (MainStorage storage $) { assembly { $.slot := LOC } }
}
```

Relevance

If the substrate selection (see `../synthesis/substrate-fork.md`) favours ERC-6909 with implementer-defined semantics rather than ERC-3475, the per-tranche net asset value, seniority index, and coverage-test state require an upgrade-safe storage location keyed by tranche identifier. ERC-7201 provides exactly this layout, and is consistent with the diamond (EIP-2535) patterns used elsewhere in the broader workstream.

Security-Token and Compliance Branch

These standards govern eligibility to hold a token — identity, investor qualification, and transfer restrictions. They are adjacent to tranching rather than part of it: they compose alongside a waterfall but do not specify one. They are relevant only where the target is a regulated or real-world-asset structure.

ERC-1400 and ERC-1410 — unofficial

These are not official EIPs. They were opened as GitHub issues by Adam Dossa of Polymath in September 2018: issue #1411 (ERC-1400) and issue #1410 (ERC-1410). Both are closed and labelled stale. The canonical specification is maintained in a community repository rather than in the Ethereum Foundation repository: `SecurityTokenStandard/EIP-Spec`.

ERC-1400 is an umbrella over five sub-standards: ERC-1594 (core transfer), ERC-1410 (partitions), ERC-1643 (document management), and ERC-1644 (controller operations). The tranche-relevant element is ERC-1410's partition model: balances are divided into `bytes32` partitions, and `canTransferByPartition` returns a status reason code. This corresponds to a tranche-as-partition representation.

ERC-3643 — T-REX — specification (Final, 2021-07-09; requires ERC-20, ERC-173)

The finalised and most widely adopted member of this branch. It specifies permissioned, compliant transfer with on-chain identity (ONCHAINID) for investor eligibility. The specification does not address tranching, seniority, or waterfall logic; it governs which parties may hold tranche tokens, not how a structure is tranched.

ERC-7518 — DyCIST — specification (Review, 2023-09-14; requires ERC-165, ERC-1155)

A more recent successor to the ERC-1400/1410 line, built over ERC-1155 partitions with off-chain compliance vouchers, token locking, and forced transfers.

Summary

The ERC-1410 partition model is useful as prior art rather than as a citable standard. For a finalised compliance layer, ERC-3643 is the appropriate standard to compose with, and ERC-7518 is the relevant proposal to track. None of these standards specifies the engine described in `../00-thesis-and-direction.md`.

Financial Mechanics

This section describes the behaviour that the existing standards do not specify, and which the proposed standard is intended to define.

- `waterfall-tranching-primer.md` — Subordination, priority of payments, and loss allocation, from first principles
- `oc-ic-coverage-tests.md` — The overcollateralisation and interest-coverage tests and their active-cure mechanic
- `heuristics-vs-plumbing.md` — The distinction between deterministic logic and parameterised calibration

Waterfall Tranching — First Principles

Structure

A pool of capital generates income and may incur losses through default. Investors have differing risk preferences. Tranching serves these preferences from a single pool by arranging claims into priority layers.

```
Senior  (e.g. 80M)  — lower yield, paid first, absorbs loss last
Junior  (e.g. 20M)  — higher yield, paid last,  absorbs loss first
      the pool sits beneath both layers
```

Direction of flows

- **Cash flow descends.** Income fills the senior claim first; the remainder is distributed to the junior claim.
- **Loss ascends.** A default reduces the junior layer first; the senior layer incurs loss only after the junior layer is exhausted.

Two distinct waterfalls typically operate in parallel: an interest waterfall and a principal waterfall.

```
income → [1. senior coupon, paid in full] → [2. junior receives the remainder]
```

Worked example

Consider a 100M pool funded as 80M senior and 20M junior.

- **No defaults, 8M of interest.** The senior coupon of 4M is paid; the junior layer receives the remaining 4M, an approximate 20% return on 20M.
- **15M of defaults.** The junior layer absorbs the loss first, reducing it to 5M; the senior layer remains whole at 80M. If a coverage test is breached, junior distributions are additionally suspended and redirected to de-lever the senior layer — see `oc-ic-coverage-tests.md`.

Definition

The junior tranche earns the larger return in favourable conditions and absorbs loss first in adverse conditions; the senior tranche earns a smaller, steadier return and is protected. Coverage tests are the conditions that enforce this protection as the pool deteriorates.

On-chain status

Surveyed on-chain protocols reproduce the layered structure (senior and junior tokens) but omit the coverage tests: their junior layer absorbs loss passively, with no redirection of cash flow. This omission defines the scope addressed by the proposed standard; see `../protocols/reference-architecture.md`.

Overcollateralisation and Interest-Coverage Tests

The two coverage tests of collateralised-loan-obligation and securitisation practice convert a waterfall from a passive distribution into an active one. This mechanic is absent from the surveyed on-chain protocols.

Overcollateralisation (OC) test

A test of whether sufficient collateral principal supports the senior claim.

```
OC ratio = (par value of the collateral pool) / (par value of senior + this tranche)
Example:  100M / 80M = 1.25, above a 1.10 trigger
```

This is a principal, or asset-value, test.

Interest-coverage (IC) test

A test of whether the pool generates sufficient income to service the senior coupon.

```
IC ratio = (interest collected) / (interest due to the senior claim)
Example:  8M / 4M = 2.0, above a 1.20 trigger
```

This is an income test.

The active-cure mechanic

On breach of either test, the waterfall re-routes cash flow: distributions that would otherwise reach the junior or equity layer are redirected to repay senior principal until the ratio returns above its trigger, which cures the test.

```
Compliant:  senior coupon → junior remainder
Breached:   senior coupon → de-lever senior (junior distributions suspended) → cure
```

In stress, the senior layer therefore receives a smaller distribution while the junior layer absorbs the shortfall first — the protection that tranching is intended to provide.

On-chain status

Behaviour	Present on-chain
Passive loss absorption (junior net asset value declines)	Yes, generally
Suspension of new senior issuance on breach	Centrifuge only
Periodic OC/IC computation with cash-flow diversion to cure	None
Excess-spread capture (residual trapped before equity)	Approximated only by Goldfinch's fixed 20% allocation
Three-or-more-tranche sequential waterfall	Attempted by Waterfall DeFi (now defunct)

Evidence is collected in `../protocols/loss-tranching-protocols.md`. The normative core of the proposed standard is the sequence: compute the coverage ratios each epoch, evaluate them against governable triggers,

and re-route the waterfall on breach. The triggers are calibration parameters rather than constants; see `heuristics-vs-plumbing.md`.

Deterministic Logic versus Parameterised Calibration

A waterfall comprises two layers. Conflating them is a design error.

Deterministic layer

The priority of payments — pay the senior tranche, distribute the remainder to the junior tranche, and divert distributions to cure a breach — is arithmetic and ordering. For a given set of inputs the result is exact and reproducible. This layer is the appropriate subject of a normative on-chain specification, because any party can verify that distributions followed the stated rules.

Calibration layer

Every threshold and parameter that feeds the deterministic layer is a modelling judgement rather than a determinable constant:

- The overcollateralisation trigger (for example 1.10) and the interest-coverage trigger (for example 1.20).
- The attachment and detachment points that set the senior-to-junior division.
- The assumed default correlation that underlies the senior layer's protection.

These parameters are conditional estimates: the senior layer is protected only while losses remain within the junior cushion and defaults do not correlate more strongly than assumed. The 2008 failure of the Gaussian-copula approach illustrates the consequence — a correlation assumption that held in calm conditions and failed when defaults correlated, causing senior tranches rated as low-risk to incur losses. The failure was one of opaque calibration, not of the payment mechanism.

Governing rule

Specify the deterministic layer precisely. Expose every calibration value as an explicit, governable, and auditable parameter, rather than embedding it as a fixed constant. The advantage available on-chain, relative to traditional practice, is that the calibration becomes public and inspectable rather than embedded in a rating and a private indenture.

Mapping to the specification

- **Specification** — the deterministic engine and the coverage-test interface.
- **Rationale and Security Considerations** — the treatment of calibration as parameterised judgement, with reference to the prior-art failure modes in `../synthesis/open-questions-and-next.md`.
- **Conformance vectors** — deterministic scenarios; calibration is left to the deployer.

On-Chain Prior Art

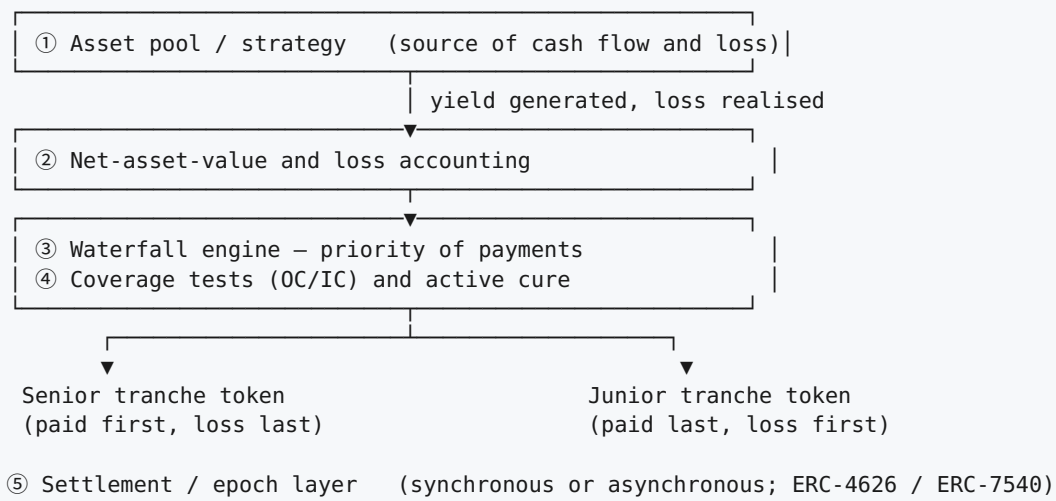
Each surveyed protocol implements a subset of the reference architecture. The purpose of this section is to identify which components each protocol provides, and to confirm that the active coverage-test component is absent across all of them.

- `reference-architecture.md` — The abstract reference architecture against which protocols are assessed
- `loss-tranching-protocols.md` — Risk axis: Idle, BarnBridge, Centrifuge, Goldfinch, Maple, Saffron, Waterfall DeFi
- `cashflow-tranching-protocols.md` — Cash-flow axis: Pendle, Spectra
- `rwa-securitization-entrants.md` — Recent entrants (2025–2026): Untangled, Galaxy CLO, Tradable
- `standards-adoption-matrix.md` — The standards each protocol adopts

The underlying sourced survey is `01-tranching-deep-research.md`, Part IV, in the research workspace.

Reference Architecture

The following is the abstract architecture against which the surveyed protocols are assessed. Each protocol is evaluated against components ① through ⑤. Component ④ is absent in all of them.



Mapping to standards

- ① — A strategy adapter, normalised through EIP-5115 SY or an ERC-4626 source.
- ② — Net-asset-value and loss recognition. This component carries the principal trust assumption and is not standardised; see `../synthesis/open-questions-and-next.md`.
- ③ **and** ④ — The engine, which the proposed standard defines. See `../mechanics/oc-ic-coverage-tests.md`.
- **Tokens** — The substrate selection between ERC-3475 and ERC-6909 with ERC-7201; see `../synthesis/substrate-fork.md`.
- ⑤ — Settlement, composing ERC-7540; the epoch boundary is the point at which component ③ executes.

Composite assessment

The closest existing assembly combines Idle's `IdleCDO` engine and dynamic split, Centrifuge's subordination-ratio gate, ERC-7540 settlement, and Goldfinch's fixed excess-spread allocation. Together these approximate the reference architecture with the exception of component ④, the active diversion of cash flow on a coverage breach, which no surveyed protocol implements. Detail in `loss-tranching-protocols.md`.

Loss-Tranching Protocols (Risk Axis)

Protocols implementing senior-and-junior subordination, the axis the proposed standard primarily addresses. Each is assessed against the reference architecture, where ② denotes net-asset-value and loss accounting, ③ the waterfall, ④ the active coverage tests, and ⑤ the settlement layer.

Idle Perpetual Yield Tranches

```
strategy → IdleCDO (tracks virtual price) → split by trancheAPRSplitRatio
        → AA token (ERC-20, senior)   BB token (ERC-20, junior, first-loss)
```

The BB tranche absorbs loss first; the AA tranche is affected only after the BB tranche's total value is exhausted, giving strict subordination. The `trancheAPRSplitRatio` adjusts the yield division as a function of the senior share of total value, which is a pricing mechanism rather than a coverage test. Provides ②, ③ (passive), and ⑤ (perpetual, ERC-4626-compliant); does not provide ④. Sources: README, Adaptive Yield Split. A full implementation-level review is in `../code-study/idle-finance/`.

BarnBridge SMART Yield (discontinued)

The senior claim was issued as sBONDs (ERC-721; fixed, dated, and guaranteed) and the junior claim as jTokens (ERC-20; residual, first-loss), with a moving-average yield oracle setting the senior rate. The structure is notable for representing seniority as a non-fungible, dated instrument and juniority as a fungible, perpetual one. The protocol was discontinued in July 2023 and was the subject of a Securities and Exchange Commission settlement in December 2023 — a regulatory outcome rather than a technical failure. Source: SPEC.md.

Centrifuge (DROP / TIN)

DROP (ERC-20, senior) and TIN (ERC-20, junior, first-loss) are issued against a pool valued as off-chain real-world-asset net asset value plus an on-chain reserve. A minimum-subordination-ratio test halts new DROP issuance when the TIN share falls below a floor, until the share is restored. This is the closest on-chain analogue to component ④, but the response is passive (issuance is suspended) rather than active (cash flow is diverted to de-lever the senior claim); that distinction is the gap the proposed standard addresses. Version 3 reissues the tranches as ERC-7540 vaults, a standard Centrifuge co-authored. Provides ②, ③, partial ④, and ⑤. Sources: A Tale of Two Tokens, RWA token standards. A full implementation-level review is in `../code-study/centrifuge/`.

Goldfinch

The Senior Pool issues FIDU (ERC-20); Backers occupy the junior, first-loss position and hold PoolTokens (ERC-721). A leverage model sizes the senior position as a multiple of the junior, and a fixed 20% of senior nominal interest is reallocated to the junior position. This reallocation is the closest on-chain analogue to traditional excess-spread capture. Provides ② and ③; does not provide ④. Source: Backers documentation.

Maple

Lender positions are ERC-4626 shares. The loss order is writedown, then borrower collateral, then liquidation of the cover (junior) position ahead of lenders, then recoveries. Cover is thin — on the order of 0–3% and often unused — which demonstrates that on-chain first-loss capital is structurally smaller than the 8–12% equity typical of traditional structures. This is a constraint for the specification rather than a mechanic. Source: defaults and impairments.

Saffron and Waterfall DeFi (dormant / defunct)

- **Saffron.** AA, A, and an auto-balancing S tranche, on 14-day epochs, with the junior position staking SFI as insurance at approximately ten-times leverage. Now dormant. Source: introduction.
- **Waterfall DeFi.** A three-tranche structure (Senior, Mezzanine, Junior) with cash flow distributed top-down and loss absorbed bottom-up, on 7-day epochs. Now defunct (CertiK score 2.1/10). It demonstrates that a three-or-more-tranche sequential waterfall is implementable but was not sustained. Source: what is tranching.

Assessment

The surveyed protocols provide non-decreasing or checkpointed net-asset-value accounting, strict junior-first loss exhaustion, dynamic coverage pricing, and subordination gating (Centrifuge). They do not provide active coverage-test cures, excess-spread capture beyond a fixed allocation, sustained three-or-more-tranche waterfalls, or adequately thick first-loss capital. See `../mechanics/oc-ic-coverage-tests.md`.

Cash-Flow-Tranching Protocols (Cash-Flow Axis)

Protocols that split a yield asset into a principal component and a yield component, rather than into senior and junior risk layers. The structure parallels the risk axis. The relevant token standards are described in `../standards/vault-and-yield-layer.md`.

Pendle

```
yield asset → SY (EIP-5115, an ERC-4626 superset wrapper)
            → split → PT (principal, redeems 1:1 at maturity)
                      YT (variable yield to maturity)
```

The principal token (PT) and yield token (YT) are both ERC-20. SY normalises the underlying yield source, and the split operates only against the SY `exchangeRate()`. The internal pyIndex is designed not to decrease, so yield volatility is absorbed by the YT first and the PT is ring-fenced; the PT therefore occupies a position analogous to the senior tranche and the YT a position analogous to a first-loss claim on yield. The structure has a fixed maturity, and the PT/YT oracle takes the maximum of the SY and PY indices to resist downward manipulation. Pendle's principal tokens implement ERC-4626 rather than EIP-5095. Sources: Pendle documentation, Minting, mixbytes analysis.

Spectra

The same principal-and-yield split applied to interest-bearing tokens: the principal token provides a fixed return at maturity and the yield token supports speculation on or hedging of future yield. Source: spectra-core.

Relevance

The non-decreasing index is a net-asset-value technique applicable to the proposed standard, and the normalise-then-split architecture supports the use of a uniform yield adapter beneath the engine (see `../standards/vault-and-yield-layer.md`). As on the risk axis, the split engine itself is unspecified, while only the token legs received standards; see `../00-thesis-and-direction.md`.

Real-World-Asset Securitisation Entrants (2025–2026)

Recent entrants closest in intent to a full waterfall. They treat loss recognition (component ②) as a first-class element, which most DeFi designs do not.

Untangled Finance

```
credit pool and Credit Oracle → SOT (Senior Obligation Token) / JOT (Junior Obligation Token)
```

Untangled uses explicit securitisation terminology and a Credit Oracle, which represents a direct attempt to provide on-chain loss recognition. It is the most traditionally-structured of the on-chain models surveyed. The token standard is not yet verified and is likely ERC-20. Source: documentation.

Galaxy CLO 2025-1

A tokenised collateralised loan obligation (senior tranche at SOFR plus 570 basis points; initial 75M, expandable to 200M; on Avalanche). The waterfall operates off-chain within the traditional structure, and the token represents the claim. This is useful as a reference for the form of a real collateralised-loan-obligation tranche rather than as on-chain mechanics. Source: PRNewswire.

Tradable

Tokenised private credit on ZKsync, following the same wrapper pattern of an off-chain waterfall and an on-chain claim. Likely permissioned (ERC-3643 or ERC-1400 family); not verified. Source: Chainlink analysis.

Assessment

These wrappers demonstrate demand for tokenised tranches but retain the waterfall off-chain; they tokenise the output of a traditional structure rather than specifying the engine. Untangled is the exception in bringing loss recognition on-chain. The token standards used by Untangled, Galaxy, and Tradable remain to be confirmed; see `standards-adoption-matrix.md` and `../synthesis/open-questions-and-next.md`.

Standards Adoption Matrix

The standards each surveyed protocol adopts, drawn from the protocol notes.

Protocol	Standards adopted	Note
Idle PYT	ERC-20 (AA/BB) and ERC-4626	Two ERC-20 tokens; ERC-4626 accounting
BarnBridge	ERC-20 (jTokens) and ERC-721 (sBONDs)	Senior claim issued as a non-fungible bond
Centrifuge	ERC-20 (DROP/TIN); ERC-7540 in version 3 (over ERC-4626)	The only ERC-7540 adopter, which it co-authored
Goldfinch	ERC-20 (FIDU) and ERC-721 (PoolTokens)	Junior position issued as non-fungible
Maple	ERC-20 and ERC-4626	Standard vault shares
Pendle	ERC-20 (PT/YT) and EIP-5115 (SY)	Principal token implements ERC-4626, not EIP-5095
Saffron, Waterfall DeFi	ERC-20	Plain fungible tokens
Untangled	ERC-20 (SOT/JOT), unverified	Likely plain ERC-20
Galaxy, Tradable	Unverified; likely ERC-3643 or ERC-1400	Permissioned real-world-asset wrappers

Observations

1. **ERC-20 predominates.** Each tranche is generally represented as a separate ERC-20 contract, the minimal representation with no shared accounting.
2. **ERC-3475 is not adopted.** The standard designed specifically to represent tranche-like instruments has minimal production adoption; protocols instead reconstruct the equivalent with ERC-20 pairs.
3. **ERC-1155 and ERC-6909 are not used for tranches.** Separate ERC-20 tokens are preferred for composability with decentralised exchanges and lending markets.
4. **ERC-4626 is the most widely adopted settlement base; ERC-7540 has a single adopter** (Centrifuge).
5. **Two standards outside the core sequence appear in practice:** ERC-721, for the dated or non-fungible tranche leg, and EIP-5115, as Pendle's yield wrapper.

Implication

The standards best-suited to tranching (ERC-3475, ERC-1155/6909, ERC-7540, EIP-5095) are largely unused in this role; the prevailing practice combines ERC-20 with ERC-4626 and a bespoke engine. Both a tranche-token standard with practical adoption and a waterfall interface therefore remain unaddressed. The non-adoption of ERC-3475 and EIP-5095 informs the substrate selection in `../synthesis/substrate-fork.md`.

Code Study

Implementation-level studies of specific protocol codebases. This section is distinct from `../protocols/`: the protocols section is a conceptual survey assessing each design against the reference architecture, whereas this section examines the source code of a single protocol in depth and maps its concrete mechanisms onto the clauses of the proposed standard.

Each protocol studied is given its own subdirectory, so that the section scales as further codebases are reviewed.

Modules

- `idle-finance/` — Idle Finance: the `IdleCDO` engine, the Adaptive Yield Split, and the Ethena (sUSDe) integration.
- `centrifuge/` — Centrifuge: the Tinlake lender module (Assessor + Coordinator subordination logic) and the v3 ERC-7540 layer.

Planned

- Pendle (`pendle-core-v2-public`) — the SY wrapper and the principal/yield split engine.

Idle Finance — Code Study

An implementation-level review of Idle Finance's Perpetual Yield Tranches, the closest production analogue to the engine the proposed standard specifies. Repository: `Idle-Labs/idle-tranches`.

The study is organised so that each concern is treated in isolation and mapped, in the final note, onto the clauses of `../../../spec/erc-waterfall-tranche.md`.

- `overview.md` — The protocol, its current scale, the Pareto transition, and the repository structure
- `idle-cdo-engine.md` — The `IdleCDO` engine: tranche tokens, net-asset-value accounting, deposit and withdrawal, and loss allocation
- `adaptive-yield-split.md` — The Adaptive Yield Split: a coverage-priced dynamic yield division
- `ethena-integration.md` — The sUSDe integration: native staking, the redemption cooldown, and the per-request clone pattern
- `mapping-to-standard.md` — How each mechanism maps onto, and differs from, the proposed standard

Summary judgement

Idle implements components ②, ③, and ⑤ of the reference architecture: net-asset-value accounting by a checkpointed virtual price, a senior-and-junior split with strict junior-first loss exhaustion, and perpetual or epoch-based settlement. Its most advanced feature, the Adaptive Yield Split, prices the protection that the junior tranche extends to the senior tranche, but it does so on the yield-distribution axis only. It contains no coverage test and no active diversion of cash flow (component ④), which remains the increment the proposed standard introduces.

Idle Finance — Overview

The protocol

Idle Finance is an Ethereum yield-automation protocol. It originated the on-chain Perpetual Yield Tranches model, in which a single yield strategy is divided into a senior tranche (AA) and a junior tranche (BB) over one underlying asset. The senior tranche receives a protected, lower yield; the junior tranche receives a higher yield and absorbs loss first. The protocol is the most architecturally complete two-tranche engine in production and is therefore the primary reference for this study, independent of its current scale.

Current scale

The protocol is small by current total value locked and is past its earlier peak. Figures retrieved 2026-06.

Metric	Value	Source
Total value locked	Approximately 3.6 million USD (DefiLlama); approximately 4.77 million USD (Stelareum)	DefiLlama, Stelareum
Chain concentration	Approximately 98.6% on Ethereum	DefiLlama
IDLE token	Micro-capitalisation (ranked approximately 9806)	CoinGecko

The Pareto transition

Idle is rebranding to Pareto (token PAR), repositioning as a credit-coordination protocol for institutional, real-world-asset private credit, with reported committed value in excess of 25 million USD on the new direction. Sources: Pareto governance — Rebranding Idle to Pareto, pareto.idle.finance.

This transition is itself relevant to the proposed standard. The migration of an originator of on-chain tranching toward structured institutional credit is consistent with the direction of Centrifuge, Maple, and the recent securitisation entrants surveyed in `../../protocols/rwa-securitization-entrants.md`, and supports the case for a waterfall standard oriented toward credit.

Repository structure

The tranche system is implemented in `Idle-Labs/idle-tranches`. The components relevant to this study are:

- `contracts/IdleCDO.sol` — the engine.
- `contracts/IdleCDOTranche.sol` — the senior and junior tranche token.
- `contracts/IdleCDOStorage.sol` — the storage layout.
- `contracts/IdleCDOFactory.sol` — the deployment factory.
- `contracts/IdleCDO<Strategy>Variant.sol` — per-strategy variants, each binding the engine to a specific strategy and underlying. Observed variants include Best Yield, Amphor, Epoch, Ethena, Gearbox, Instadapp Lite, Leveraged Euler, PoLido, Truefi, and Usual.
- `contracts/strategies/<protocol>/` — the strategy adapters, for example `strategies/ethena/EthenaSusdeStrategy.sol`.

Each variant follows one rule: the engine's underlying asset is the base asset of the strategy it wraps. The protocol therefore has no single underlying asset; each deployment is denominated in its strategy's base asset, predominantly major stablecoins, with some liquid-staking variants. The Ethena variant, examined in `ethena-integration.md`, is denominated in USDe.

The IdleCDO Engine

The engine is `contracts/IdleCDO.sol`. It holds the strategy position, accounts for value through a per-tranche virtual price, and divides yield between the two tranches. This note describes its structure at the level supported by the repository's documentation and the variant overrides examined for this study; precise arithmetic should be confirmed against the source.

Components

Contract	Role
`IdleCDO.sol`	The engine: deposit, withdrawal, harvest, price update, and yield split.
`IdleCDOTranche.sol`	The tranche token. Each of the senior (AA) and junior (BB) tranches is a minimal ERC-20 minted and burned by the engine.
`IdleCDOStorage.sol`	The storage layout, separated from logic to support upgradeability.
`IdleCDOFactory.sol`	Deployment of new instances.
`IdleCDO<Strategy>Variant.sol`	Per-strategy variants overriding deposit, withdrawal, or harvest as a strategy requires.

Tranche representation

The senior and junior tranches are two independent ERC-20 contracts (`IdleCDOTranche`), minted on deposit and burned on withdrawal. This is the plain "one ERC-20 per tranche" representation noted in `../../standards/token-substrate-layer.md`: the engine, not the token, carries the seniority and accounting logic.

Net-asset-value accounting

The engine maintains a virtual price for each tranche, updated on harvest, representing the value of one tranche token in the underlying asset. Gains raise the virtual price; losses reduce it. The senior and junior virtual prices diverge according to the yield split (see `adaptive-yield-split.md`) and the allocation of loss. This checkpointed virtual price is component ② of the reference architecture: value is recognised internally from the strategy's reported position, an assumption discussed under loss recognition in `../../synthesis/open-questions-and-next.md`.

Deposit and withdrawal

On deposit, the engine mints tranche tokens against the deposited underlying and routes the underlying to the strategy. The Ethena variant illustrates the pattern with a minimal override:

```
function _deposit(uint256 _amount, address _tranche, address _referral)
  internal override whenNotPaused returns (uint256 _minted) {
  _minted = super._deposit(_amount, _tranche, _referral);
  IIdleCDOStrategy(strategy).deposit(_amount);
}
```

On withdrawal, the engine burns tranche tokens and returns the underlying, recovering it from the strategy where necessary. For strategies whose redemption is not immediate, the variant defers the exit; see `ethena-integration.md`.

Loss allocation

Loss is absorbed by the junior tranche first. The senior tranche is affected only after the entire junior tranche value is exhausted, which gives strict subordination. There is one ordered boundary between two tranches, and the protection is passive: a loss reduces the junior virtual price directly, with no test evaluated and no cash flow redirected. The active redirection of cash flow on a coverage breach, present in traditional structures, is absent; this is the subject of `mapping-to-standard.md`.

The variant pattern

The engine is bound to a yield source through a strategy adapter and a thin `IdleCDO<Strategy>Variant`. The adapter presents a uniform interface (`IIdleCDOStrategy`) over a specific protocol, and the variant overrides only the methods that the strategy's mechanics require. This separation of a stable engine from a per-source adapter is the same composition principle the proposed standard adopts in placing a yield adapter beneath the engine; see `../../protocols/reference-architecture.md`.

Adaptive Yield Split

The Adaptive Yield Split is the yield-distribution rule inside `IdleCDO`, applied at harvest when a period's yield is divided between the senior (AA) and junior (BB) tranches. It is the most advanced feature of the engine. Primary source: Idle — Adaptive Yield Split fosters PYTs' liquidity scalability and the Adaptive Yield Split documentation.

The prior fixed split and its limitations

Under a fixed yield split, risk and reward were misaligned in four respects, per the source article:

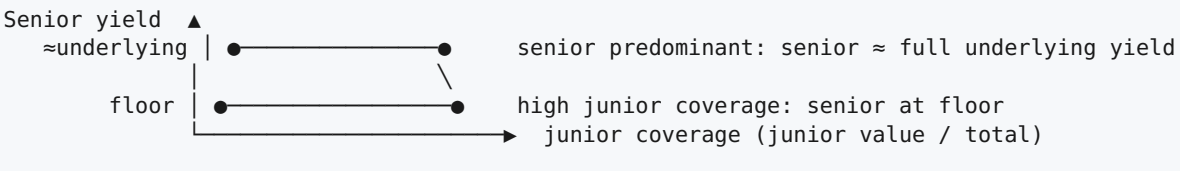
1. The senior tranche was underpaid, receiving "a yield lower than half of the underlying (5% APY) in most cases".
2. Coverage was asymmetric: when junior liquidity was low, the senior tranche surrendered yield for protection that was not yet present.
3. The junior tranche had a scalability ceiling, since a fixed split diluted junior returns below the underlying yield beyond a certain size.
4. Governance-token incentives concentrated in a single senior pool.

The mechanism

The split is made a function of the coverage ratio — the relative size of the senior and junior tranches. In the engine this is the `trancheAPRSplitRatio`, the share of the pool's yield allocated to the senior tranche, recomputed from the senior-value ratio:

Pool state	Senior share of yield	Effect
Low junior coverage (senior predominant)	High, approaching approximately 99%	The senior tranche earns close to the full underlying yield.
High junior coverage (large junior tranche)	Reduced, but floored at approximately 50%	The senior yield falls toward a guaranteed floor; the junior tranche captures up to half.

The relationship is monotonic: the more protection the junior tranche provides, the more yield the senior tranche cedes in exchange, subject to a floor, while the junior tranche is designed always to earn above the underlying yield.



The source article gives a worked illustration: a 2.5 million USD senior position at a 10% underlying yield, with a 30% governance-incentive allocation, realises approximately 7% when the senior tranche is fully covered by an equal junior balance and up to approximately 12% when the senior tranche predominates.

The approximately 99% cap and approximately 50% floor on `trancheAPRSplitRatio` are properties of the implementation and the Adaptive Yield Split documentation; the article describes the behaviour qualitatively and states no closed-form expression.

Effect on liquidity and scalability

- The senior tranche earns a competitive organic yield independent of junior deposits, which removes the requirement to accumulate coverage before offering competitive returns.
- The junior tranche accepts arbitrary size, because the split re-prices as the junior tranche grows rather than diluting toward the underlying yield.
- Governance incentives can be distributed across more pools, since organic senior yields are already competitive.

Risk model

The junior tranche is the loss-absorption layer for the senior tranche, taking loss first. The Adaptive Yield Split prices that protection: when junior coverage is high, the senior tranche accepts a lower, floored yield reflecting the protection then in place. The source identifies the mechanism as most valuable for higher-risk, higher-yield strategies, citing leveraged staked ether, where the senior tranche obtains a base yield while its capital is protected from liquidation risk.

Classification

The Adaptive Yield Split is a yield-pricing mechanism, not a coverage test and not a waterfall. It adjusts the division of yield as a continuous function of coverage, but it does not redirect principal cash flow and contains no trigger that diverts cash flow to cure a breach; loss absorption remains passive. This classification is the central observation carried into `mapping-to-standard.md`.

Ethena (sUSDe) Integration

The Ethena variant illustrates how Idle binds the engine to a yield source whose redemption is not immediate, and does so without any decentralised-exchange swap. The integration comprises four files in `Idle-Labs/idle-tranches`:

- `contracts/IdleCDOEthenaVariant.sol`
- `contracts/strategies/ethena/EthenaSusdeStrategy.sol`
- `contracts/strategies/ethena/EthenaCooldownRequest.sol`
- `contracts/interfaces/ethena/IStakedUSDeV2.sol`

Underlying and strategy token

The engine's underlying asset is USDe; the strategy token is sUSDe, Ethena's `StakedUSDeV2`, which is itself an ERC-4626 vault whose asset is USDe. Confirmed addresses, from the strategy test fixture:

```
address internal constant USDe  = 0x4c9EDD5852cd905f086C759E8383e09bff1E68B3; // underlying / deposit asse
address internal constant SUSDe = 0x9D39A5DE30e57443BfF2A8307A4256c8797A3497; // strategy token (StakedUSD
address internal defaultUnderlying = USDe;
```

The strategy is a thin extension of `ERC4626Strategy`:

```
contract EthenaSusdeStrategy is ERC4626Strategy {
  function initialize(address _vault, address _underlying, address _owner) public {
    _initialize(_vault, _underlying, _owner);
  }
}
```

Because the underlying equals the strategy's base asset (USDe), the engine never converts between assets. Any conversion — for example from USDC to USDe — is performed by the depositor or a front-end before deposit, outside the engine, and bears its own liquidity and slippage risk. This is the single-asset denomination principle: the engine operates entirely in one asset, and conversion is external.

Deposit: native staking

On deposit, the underlying USDe is staked into sUSDe through the strategy. No swap occurs.

```
function _deposit(uint256 _amount, address _tranche, address _referral)
  internal override whenNotPaused returns (uint256 _minted) {
  _minted = super._deposit(_amount, _tranche, _referral);
  IIdleCDOStrategy(strategy).deposit(_amount); // USDe -> sUSDe
}
```

Redemption: the native cooldown

sUSDe cannot be redeemed immediately; `StakedUSDeV2` enforces a per-address cooldown, after which the position is unstaked to USDe. The variant uses this native path rather than a swap, and manages the per-address constraint by deploying a minimal-proxy clone for each withdrawal:

```
EthenaCooldownRequest clone = EthenaCooldownRequest(
  cooldownImpl.clone(abi.encodePacked(address(this), msg.sender), msg.value));
IERC20Detailed(strategyToken).safeTransfer(address(clone), SUSDeRedeemed);
clone.startCooldown();
```

The clone (`EthenaCooldownRequest` , implementation `0xe0C4a2B14F0ACd936226A598BE6BfeD190E098d1`) holds the sUSDe for one request and exposes:

```
function startCooldown() external { require(msg.sender == _getCDO(), '6');
  IStakedUSDeV2(SUSDE).cooldownShares(/* full sUSDe balance */); }
function unstake() external { IStakedUSDeV2(SUSDE).unstake(_getUser()); } // sends USDe to the user
function rescue(address _token) external; // timelock multisig only
```

The CDO address and the user address are stored as immutable arguments in the clone's bytecode, so `unstake()` delivers USDe directly to the original withdrawer after the cooldown elapses.

Rationale for the clone pattern

Ethena's cooldown is keyed per address, with one cooldown slot per holder. Routing every withdrawal through a single contract would cause a new request to overwrite a prior request's timer and amount. A clone per withdrawal gives each request an isolated cooldown slot at minimal deployment cost.

Consequence for settlement

The integration trades an asset-conversion problem for a redemption-latency problem: exits are not immediate but are deferred behind the cooldown. An asset with redemption latency cannot settle synchronously, which is the practical case for composing an asynchronous, request-based settlement layer. This is direct evidence for the settlement choice in the proposed standard; see `mapping-to-standard.md` and `../../standards/vault-and-yield-layer.md`.

Mapping to the Proposed Standard

This note maps Idle's mechanisms onto the components of the reference architecture and the clauses of `../../../spec/erc-waterfall-tranche.md`, and states precisely what the standard adds.

Component mapping

Component	Idle mechanism	Standard clause
① Strategy	`IIdleCDOStrategy` adapter and `IdleCDO<Strategy>Variant`	Composed yield source, outside the interface
② Net-asset-value and loss accounting	Per-tranche virtual price, updated on harvest	`totalAssets()`, `trancheAssets()`
③ Waterfall	Two-tranche split; strict junior-first loss exhaustion	`distribute()`, `allocateLoss()`
④ Coverage tests and active cure	None	`coverageRatio()`, `isBreached()`, `Diversion`
⑤ Settlement	Perpetual, or epoch-based in the Epoch and Ethena variants	Composition with ERC-4626 and ERC-7540
Tranche tokens	`IdleCDOTranche` (one ERC-20 per tranche)	`trancheToken()`

Two precise distinctions

The Adaptive Yield Split is a pricing mechanism, not a coverage test. It adjusts `trancheAPRSplitRatio`, the division of yield, as a continuous function of coverage. It does not measure a ratio against a trigger, and it does not redirect principal cash flow on a breach. In the standard's terms it has no analogue of `isBreached()` or the `Diversion` event; it operates on the yield-distribution axis only. The standard's coverage tests are orthogonal to it, and the two could coexist: a deployment could price yield by an Adaptive-Yield-Split-style rule while also enforcing coverage tests that divert cash flow.

Loss absorption is passive. A loss reduces the junior virtual price directly. There is no point at which cash flow that would reach the junior tranche is instead applied to de-lever the senior tranche. This is the increment the standard introduces in clause `distribute()` rule 3 and the `Diversion` event.

What the standard adds beyond Idle

1. **An ordered, n-tranche model.** Idle implements one senior-junior boundary. The standard specifies an arbitrary number of tranches under a strictly ordered seniority index.
2. **Coverage tests with active cure.** The overcollateralisation and interest-coverage tests, and the diversion of cash flow on breach, which Idle does not implement.
3. **Governable, auditable calibration.** Triggers exposed as parameters with events, per `../../mechanics/heuristics-vs-plumbing.md`.

What the standard should adopt from Idle

1. **The checkpointed virtual price** as a concrete, workable form of component ②.
2. **The strict junior-first exhaustion rule**, which the standard generalises to reverse-seniority loss allocation.
3. **The engine-and-adapter separation**, which keeps the engine asset-agnostic and is the basis for the single-asset denomination confirmed in `ethena-integration.md`.
4. **The evidence for asynchronous settlement.** The Ethena cooldown demonstrates that redemption latency is a real and recurring property of yield sources, supporting composition with ERC-7540 for the settlement layer.

Centrifuge — Code Study

An implementation-level review of Centrifuge, the on-chain real-world-asset securitisation protocol, across its two generations. Centrifuge is the design closest to the proposed standard, because it is the only surveyed protocol that enforces a subordination constraint in explicit on-chain logic.

The study is organised so that each concern is treated in isolation and mapped, in the final note, onto the clauses of `../../../spec/erc-waterfall-tranche.md`.

- `overview.md` — The two generations (Tinlake and Liquidity Pools v3), the repositories, and where to study which concern
- `tinlake-architecture.md` — The Tinlake lender module: contract roles, the inheritance/dependency base, and the per-pool deployment model
- `assessor-and-coordinator.md` — The valuation/coverage core and the epoch solver (the centerpiece)
- `liquidity-pools-v3.md` — The current ERC-7540 layer, and the consequence of moving the coverage math off-chain
- `contract-map.md` — Which contracts are relevant to tranching, what to ignore, and a reading order
- `mapping-to-standard.md` — How each mechanism maps onto, and differs from, the proposed standard

Summary judgement

Tinlake implements components ②, ③, and ⑤ of the reference architecture with the most explicit on-chain coverage logic in the survey: the **Assessor** computes a senior ratio and enforces `minSeniorRatio` / `maxSeniorRatio` bounds, and the **Coordinator** executes epoch orders subject to those bounds in a fixed priority. This is coverage expressed as a *constraint on order execution*, not the active *diversion of cash flow to cure a breach* that component ④ specifies; that distinction remains the increment the proposed standard introduces. The successor (Liquidity Pools v3) reissues tranches as ERC-7540 vaults but relocates the coverage and pricing math off-chain, which is the opposite of the verifiable-on-chain direction the standard pursues.

Related: `../idle-finance/` (the other primary code study) and the conceptual survey in `../../protocols/loss-tranching-protocols.md`.

Centrifuge — Overview

Centrifuge tokenises real-world-asset credit pools and divides each pool into a senior and a junior tranche. It exists in two generations, and the right one to study depends on the concern.

The two generations

	Tinlake (v2)	Liquidity Pools / Protocol (v3)
Chains	Single-chain (Ethereum)	Multichain
Tranche tokens	**DROP** (senior) / **TIN** (junior), restricted ERC-20	Restricted ERC-20 tranche token per vault
Settlement	Epoch-based order solving, on-chain	**ERC-7540** async vaults
Coverage / pricing math	**On-chain** (Assessor, Coordinator)	**Off-chain** (computed on the Centrifuge chain, applied on the EVM side)
Toolchain	dapptools / MakerDAO "dss" style	Foundry
Repository	`centrifuge/tinlake`	`centrifuge/liquidity-pools`, `centrifuge/protocol`

Where to study which concern

- **The tranche/coverage *mechanism*** → Tinlake. Its valuation, subordination, and epoch logic are on-chain and explicit. See `tinlake-architecture.md` and `assessor-and-coordinator.md`.
- **The ERC-7540 settlement *interface*** → Liquidity Pools v3. See `liquidity-pools-v3.md`.

The consequence for the proposed standard: the coverage logic worth learning from is in Tinlake. v3 deliberately moved that logic off-chain, which is the inverse of the standard's goal of verifiable on-chain coverage. This is examined in `mapping-to-standard.md`.

Token vocabulary (Tinlake)

Three distinct token types appear in the code and should not be conflated:

- `currency` — the pool's underlying ERC-20 stablecoin (DAI in deployments). What investors deposit and redeem into.
- **Tranche tokens** — **DROP** (senior) and **TIN** (junior); the investor's shares, priced by the Assessor.
- **Collateral** — NFTs representing the real-world loans (borrower side); not part of the tranche mechanism.

An investment deposits `currency` and receives tranche tokens. A **redeem** is a queued request to convert tranche tokens back into `currency`, fulfilled (possibly partially) at epoch close, subject to liquidity and the senior-coverage constraints. Centrifuge has **no ERC-4626 vault** in Tinlake: the protocol predates ERC-4626, and its epoch/async model cannot be expressed by a synchronous vault interface; the vault concept appears only in v3, as ERC-7540.

Tinlake Architecture

Tinlake (`centrifuge/tinlake`) follows the MakerDAO "dss" style: many small single-purpose contracts wired together at deployment, with privileged calls gated by an `auth` allowlist. A pool comprises a borrower side (loans and a NAV feed) and a lender side (the tranches). The tranche mechanism lives in `src/lender/`.

Pool value is defined as **NAV + Reserve**: the off-chain-priced value of the real-world loans plus the on-chain liquid `currency` held in the reserve.

The lender contracts (`src/lender/`)

Contract	Role
`token/restricted.sol` + `token/memberlist.sol`	The DROP (senior) and TIN (junior) ERC-20 tokens, with a memberlist gating transfers to permitted (KYC-ed) addresses.
`operator.sol`	Investor entry point: submit `supplyOrder` / `redeemOrder`, and `disburse` fulfilled amounts after an epoch.
`tranche.sol`	Per-tranche unit: aggregates supply/redeem orders, holds the tranche's currency between epochs, and mints/burns the tranche token on execution. Deployed once per tranche, so **twice per pool**.
`reserve.sol`	The liquid `currency` buffer; the on-chain half of pool value.
`assessor.sol`	Valuation and coverage: the senior ratio, its bounds, senior debt accrual, and the senior/junior token prices. See `assessor-and-coordinator.md`.
`coordinator.sol`	The epoch solver that executes orders subject to the coverage constraints. See `assessor-and-coordinator.md`.
`definitions.sol`	An abstract base holding the shared pure calculation helpers (`calcSeniorRatio`, `calcAssets`, the token-price calcs).
`deployer.sol`, `fabs/`	The factory layer (see below).

The shared base and dependency layer

Tinlake distributes its functionality across base contracts and external packages, so much of what a file uses is inherited rather than declared locally. For example:

```
contract Assessor is Definitions, Auth, Interest { ... }
abstract contract Definitions is FixedPoint, Math { ... }
```

- **`Definitions`** (`src/lender/definitions.sol`) — the pure calc helpers (`calcSeniorRatio`, `calcExpectedSeniorAsset`, `calcAssets`, token prices), inherited by both the Assessor and the Coordinator so the two compute identical values.
- **`Auth`** (`lib/tinlake-auth/src/auth.sol`) — the access-control layer: a `mapping(address => uint256) wards` (1 = authorised), with `rely` / `deny` to grant/revoke and an `auth` modifier gating every privileged function (the parameter setters, `depend`, `file`).

- **`Interest`** (`lib/tinlake-math/src/interest.sol`) — ray-based interest math (`chargeInterest`, `rpow` per-second compounding); the basis of senior debt accrual.
- **`FixedPoint`** (`src/fixed_point.sol`) — defines `Fixed27`, a struct wrapping a `uint256` interpreted as a 27-decimal fixed-point number (a "ray", where `1e27 = 1.0 = 100%`). Used for all ratios and rates.
- **`Math`** (`lib/tinlake-math/src/math.sol`) — `safeAdd` / `safeSub` / `safeMul` (manual overflow checks, since the code targets Solidity < 0.8) and the ray/wad operators `rmul` / `rdiv`.

External packages (`tinlake-math`, `tinlake-auth`, `tinlake-erc20`, `tinlake-title`) are git submodules under `lib/`, resolved through import remappings (for example `tinlake-math/ = lib/tinlake-math/src/`). They are present only after `git submodule update --init --recursive`.

Dependency injection: the `depend` pattern

A contract declares a typed, `public`, initially-unset state variable for each neighbour it calls, for example `CoordinatorLike public coordinator;` or `ERC20Like public currency;`. The address is injected at deployment by an `auth`-gated setter:

```
function depend(bytes32 what, address addr) public auth {
    if (what == "coordinator") coordinator = CoordinatorLike(addr);
    // ...
}
```

The `...Like` interfaces (`ERC20Like`, `CoordinatorLike`, `AssessorLike`) are minimal local declarations listing only the methods that contract calls on its neighbour; the implementations live in the real contracts at the injected addresses.

The per-pool deployment model

Tinlake deploys an **entire lender stack per pool** — two tranches, an assessor, a coordinator, a reserve — isolated to that one pool, so that one pool cannot affect another. This is what the factory layer is for: `src/lender/fabs/` holds a fabricator per component (`fabs/tranche.sol`, `fabs/assessor.sol`, …), and `src/lender/deployer.sol` orchestrates a deployment, calling the tranche fabricator **twice** (senior and junior) and wiring every contract together with `rely` / `depend`. The set of `rely` calls in `deployer.sol` defines the trust graph (for example, the coordinator is made a ward of the assessor, tranches, and reserve so it can drive them at epoch close).

This per-tranche-as-its-own-contract layout, with a shared coordinator and assessor above, is the opposite of Idle's single-engine layout; the design fork it implies for the proposed standard is discussed in `mapping-to-standard.md` and `../../synthesis/substrate-fork.md`.

The Assessor and Coordinator

These two contracts are the centerpiece of Tinlake and the closest existing on-chain expression of the mechanism the proposed standard targets. The Assessor values the tranches and defines the subordination constraint; the Coordinator executes each epoch's orders subject to that constraint.

Assessor (`src/lender/assessor.sol`)

State (with `Fixed27` ray-scaled ratios, `1e27 = 100%`):

- `Fixed27 public seniorRatio` — the senior tranche's share of pool value, from the last executed epoch.
- `Fixed27 public minSeniorRatio` / `maxSeniorRatio` — the bounds the senior ratio must stay within (the subordination floor and cap).
- `uint256 public seniorDebt_` and `seniorBalance_` — together the total senior asset value; the debt portion accrues interest, the balance does not.
- `Fixed27 public seniorInterestRate` — the per-second rate at which senior debt accrues (DROP's protected return).

Behaviour:

- **`calcSeniorRatio`** is inherited from `Definitions`, not declared in this file. Its formula is `rdiv(seniorAsset, calcAssets(nav, reserve))`, where `calcAssets(nav, reserve) = nav + reserve`. In words: **seniorRatio = seniorAsset ÷ (NAV + reserve)** — the senior coverage ratio.
- **`dripSeniorDebt()`** accrues `seniorDebt_` forward at `seniorInterestRate` (via the inherited `Interest` math), so the senior tranche earns a fixed, time-based return.
- **`calcSeniorTokenPrice(nav, reserve)`** and **`calcJuniorTokenPrice(nav, reserve)`** derive the per-token prices; the junior price is the **residual** after the senior claim, so the junior tranche absorbs loss first.
- **`reBalance`** / **`changeSeniorAsset`** recompute `seniorRatio` whenever the senior asset value or pool value changes; `seniorRatioBounds()` exposes the min/max bounds for the Coordinator to read.

In standard terms, the Assessor is the on-chain form of component ② (valuation) plus the *measurement* half of component ④ (the coverage ratio and its bounds).

Coordinator (`src/lender/coordinator.sol`)

The Coordinator runs the pool in epochs and decides which pending orders to execute.

- **`closeEpoch()`** — after the minimum epoch duration, it snapshots NAV, reserve, senior asset, and token prices, and reads the pending supply/redeem orders from both tranches. If the full set of orders satisfies all constraints, it executes immediately; otherwise a solution-submission window opens.
- **`submitSolution()`** — during the window, any party may propose fulfilment amounts. Solutions are scored by a weighted objective with a fixed priority: **senior redeem** > **junior redeem** > **junior supply** > **senior supply**. The best valid solution wins after a challenge period. This priority ordering is, in effect, the payment-priority of the waterfall.
- **`validate()`** — checks a candidate solution in layers:

- *core constraints*: sufficient currency available, and each fulfilment within its order amount;
- *pool constraints*: the reserve must not exceed `assessor.maxReserve()`, and the resulting senior asset must keep the senior ratio within `[minSeniorRatio, maxSeniorRatio]` (for example `seniorAsset < rmul(assets, minSeniorRatio)` is rejected);
- a closing guard: when the junior token price reaches zero (junior exhausted), only redemptions are permitted.
- **Execution** calls `epochUpdate(...)` on each tranche with the fulfilment percentages and prices, and `changeSeniorAsset(...)` on the Assessor to record the new senior position.

In standard terms, the Coordinator is the on-chain form of component ③ (the waterfall) plus the *enforcement* half of component ④ — but enforced as a **constraint on which orders may execute**, not as a diversion of cash flow to cure a breach.

The precise limit, relative to the standard

Tinlake enforces subordination by **gating order execution**: a solution that would push the senior ratio out of bounds is simply rejected, so junior redemptions are throttled to keep the senior tranche covered. It does **not** take cash that would have flowed to the junior tranche and actively apply it to de-lever the senior tranche to *cure* an existing breach. That active-cure behaviour is the increment the proposed standard adds; see `mapping-to-standard.md` and `../../mechanics/oc-ic-coverage-tests.md`.

Liquidity Pools (v3)

The current Centrifuge codebase (`centrifuge/liquidity-pools` , also `centrifuge/protocol`) reissues tranches as **ERC-7540 asynchronous vaults** and extends the protocol across chains. It is the reference for the settlement interface, not for the coverage logic.

Tranche-relevant contracts (`src/`)

Contract	Role
`token/Tranche.sol`	The tranche token: a restricted ERC-20 (the DROP/TIN equivalent).
`token/RestrictionManager.sol`	Transfer restrictions / compliance (the memberlist equivalent).
`ERC7540Vault.sol`	One async vault per tranche: `requestDeposit` / `requestRedeem` followed by later fulfilment. The settlement rail.
`InvestmentManager.sol`	Processes investment and redemption requests and applies the fulfilled prices, minting and burning tranche tokens.
`PoolManager.sol`	Registers pools, tranches, and currencies, and deploys vaults and tranche tokens via the factories.
`Escrow.sol`	Holds pending assets and shares between request and fulfilment.
`factories/ERC7540VaultFactory.sol` , `factories/TrancheFactory.sol`	Deployment of vaults and tranche tokens.

The off-chain relocation

The contracts above hold tokens, accept requests, and apply prices, but they do **not** compute the senior ratio, the NAV, or the epoch solution. That logic runs **off-chain on the Centrifuge chain**, and the results are delivered to the EVM contracts as messages through the cross-chain layer (`gateway/Gateway.sol` and its adapters). The `InvestmentManager` then applies the fulfilled prices that arrive.

The practical consequence: the explicit coverage math that is on-chain and auditable in Tinlake's Assessor and Coordinator is, in v3, **not present on the EVM side at all**. The senior/junior pricing and the subordination decision are computed by an off-chain system and trusted on arrival.

Why ERC-7540 rather than ERC-4626

Tinlake's redemption is asynchronous and liquidity-constrained: an investor submits an order and is fulfilled, possibly partially, at epoch close. A synchronous ERC-4626 vault cannot represent this. ERC-7540 is the asynchronous vault standard (`requestDeposit` / `requestRedeem` → later fulfilment), so v3 expresses Tinlake's epoch/order model through the standard built for it. This is direct evidence for composing ERC-7540 as the settlement layer beneath a tranche engine; see `../../standards/vault-and-yield-layer.md` .

Relevance to the standard

v3 confirms two things. First, the asynchronous settlement model is real and recurring, and ERC-7540 is its standardised form. Second, at the modern multichain layer no protocol keeps the coverage logic on-chain — Centrifuge moved it off-chain entirely — which is precisely the gap the proposed standard addresses by specifying the coverage engine as verifiable on-chain behaviour. See `mapping-to-standard.md`.

Contract Map — What Is Tranching, What Is Not

This note isolates the contracts relevant to the tranche mechanism in each generation, and gives a reading order.

Tinlake — for the mechanism

Tranche-relevant, all under `src/lender/`:

Contract	Role
`definitions.sol`	Shared pure calc helpers (`calcSeniorRatio`, `calcAssets`, token prices), inherited by the assessor and coordinator.
`token/memberlist.sol`, `token/restricted.sol`	The DROP/TIN tranche tokens and their transfer allowlist.
`operator.sol`	Investor entry: supply/redeem orders and disbursement.
`tranche.sol`	Per-tranche order aggregation, currency custody, and mint/burn.
`reserve.sol`	The liquid currency buffer (the on-chain half of pool value).
`assessor.sol`	Valuation, senior ratio, bounds, senior debt accrual, token prices.
`coordinator.sol`	The epoch solver enforcing the coverage constraints.

Supporting (read as needed): `src/fixed_point.sol` (`Fixed27`), `lib/tinlake-math/src/{math,interest}.sol` (ray math and interest), `lib/tinlake-auth/src/auth.sol` (`wards`/`auth`).

Out of scope for the mechanism: the borrower side and the NAV feed (the asset side that supplies `nav` to the assessor); `deployer.sol` and `fabs/*` (deployment); `admin/*` (governance setters); `adapters/mkr/*` (a MakerDAO DAI credit line).

Reading order (Tinlake)

`definitions.sol` → `token/memberlist.sol` + `token/restricted.sol` → `operator.sol` → `tranche.sol` → `reserve.sol` → `assessor.sol` → `coordinator.sol`. The mechanism, in that order: investors submit orders through the operator, stored per `tranche`; at epoch close the `coordinator` reads NAV and `reserve`, asks the `assessor` for prices and senior-ratio bounds, selects the order fulfilment that respects subordination, and calls back into each `tranche` to mint or burn.

Liquidity Pools v3 — for the settlement interface

Tranche-relevant, under `src/`: `ERC7540Vault.sol`, `InvestmentManager.sol`, `PoolManager.sol`, `token/Tranche.sol`, `token/RestrictionManager.sol`, `Escrow.sol`, `factories/{ERC7540VaultFactory,TrancheFactory}.sol`.

Out of scope for tranching: `gateway/*` and `adapters/*` (cross-chain messaging), `libraries/*`, `Root.sol`/`Auth.sol`/`admin/Guardian.sol` (access control), `CentrifugeRouter.sol` (router). Note that the coverage and pricing logic is not in these contracts at all — it runs off-chain (see `liquidity-pools-v3.md`).

Recommendation

Study **Tinlake** for the mechanism — specifically `assessor.sol` and `coordinator.sol` after the lead-up files — because that is where the subordination logic is on-chain and explicit. Use **v3** (`ERC7540Vault.sol`, `InvestmentManager.sol`) only to see the async settlement interface.

Mapping to the Proposed Standard

This note maps Centrifuge's mechanisms onto the reference architecture and the clauses of `../../../spec/erc-waterfall-tranche.md`, and states what the standard takes and what it adds.

Component mapping (Tinlake)

Component	Tinlake mechanism	Standard clause
① Strategy / asset source	Borrower loans + NAV feed	Composed source, outside the interface
② NAV / loss accounting	`assessor` valuation; pool value = NAV + reserve	`totalAssets()`, `trancheAssets()`
③ Waterfall	`coordinator` priority ordering (senior redeem > junior redeem > junior supply > senior supply)	`distribute()`, `allocateLoss()`
④ Coverage tests + cure	`assessor` senior ratio + `min/maxSeniorRatio`; `coordinator.validate()` enforces them — **as a constraint, not an active cure**	`coverageRatio()`, `isBreached()`, `Diversion`
⑤ Settlement	Epochs (`closeEpoch`); v3 reissues as ERC-7540 vaults	Composition with ERC-4626 / ERC-7540
Tranche tokens	DROP / TIN restricted ERC-20 (v3: `Tranche.sol`)	`trancheToken()`

The two precise distinctions

Coverage is a constraint, not an active cure. Tinlake's Coordinator rejects any epoch solution that would push the senior ratio outside `[minSeniorRatio, maxSeniorRatio]`, which throttles junior redemptions to keep the senior tranche covered. It does not take cash that would reach the junior tranche and apply it to repay senior principal to *cure* a breach already in progress. The standard's `distribute()` rule 3 and the `Diversion` event specify exactly that active redirection, which Tinlake does not perform. This is the single clearest "closest, but not the same" finding in the survey.

The modern layer pushed the math off-chain. Tinlake's coverage logic is on-chain and auditable. Centrifuge v3 relocated the NAV, pricing, and epoch-solving to the off-chain Centrifuge chain, leaving the EVM contracts to hold tokens and apply delivered prices. The proposed standard's premise is the opposite: keep the coverage engine on-chain and verifiable.

The deployment fork

Centrifuge and Idle take opposite layouts for the same senior/junior concept:

- **Centrifuge (Tinlake):** one contract *per tranche* (two `tranche.sol` instances) plus a shared `coordinator` and `assessor` above them; tranches are first-class contracts, deployed per pool by a factory layer.
- **Idle:** one engine (`IdleCDO`) holding both tranches as two token addresses; tranches are plain ERC-20s indexed by the engine.

This is a real interface decision for the standard: does a tranche expose its own contract (Centrifuge-style — `trancheToken()` could be a full vault), or is it an `id`/token the engine indexes (Idle-style)? It connects directly to `../../synthesis/substrate-fork.md`.

What the standard should adopt from Centrifuge

1. **The explicit senior-ratio model with governable bounds** (`seniorRatio` against `minSeniorRatio`/`maxSeniorRatio`) — the cleanest on-chain expression of subordination, and a direct model for `coverageRatio()`/`isBreached()`.
2. **The fixed-priority order of execution** (senior redeem first, senior supply last) — a concrete payment-priority ordering.
3. **Senior debt accruing a per-second rate** (`dripSeniorDebt`) — a workable form of a protected senior return.
4. **Junior price as the residual** — a clean realisation of junior-first loss.
5. **ERC-7540 composition** for the settlement layer, as v3 demonstrates the async model requires.

What the standard adds beyond Centrifuge

1. **Active cure**, not just constraint: divert cash flow to de-lever the senior tranche on a breach, rather than only rejecting order combinations that would breach.
2. **An arbitrary number of ordered tranches**, where Tinlake fixes two (DROP/TIN).
3. **On-chain, verifiable coverage at the modern layer** — the part v3 moved off-chain.
4. **An interest-coverage test** alongside overcollateralisation; Tinlake's constraint is overcollateralisation-style (a ratio of assets), with no separate income-coverage test.

Synthesis

This section records the conclusions of the research and the decisions they require, which feed into `../../spec/`.

- `substrate-fork.md` — Selection of the token substrate: ERC-3475 versus ERC-6909 with ERC-7201
- `open-questions-and-next.md` — Research questions outstanding before drafting the normative specification

The principal conclusion is stated in `../00-thesis-and-direction.md`: specify the deterministic waterfall and a coverage-test interface, parameterise the calibration, and compose ERC-4626 and ERC-7540 beneath a tranche-token substrate.

Token-Substrate Selection

The first substantive specification decision concerns the token used to represent a tranche. Two viable approaches are considered.

Option A — ERC-3475

- **In favour.** ERC-3475 provides bond semantics directly: class and nonce identity, on-chain per-series net asset value and metadata, a supply lifecycle, and a redemption-condition accessor. It is the closest existing representation of a tranche.
- **Against.** It is comparatively complex (the batched `Transaction` interface), it has minimal production adoption (see `../protocols/standards-adoption-matrix.md`), and it specifies neither a seniority ordering nor a waterfall, both of which would have to be added regardless.

Option B — ERC-6909 with ERC-7201

- **In favour.** ERC-6909 is lightweight, has an ERC-20-like interface, and has practical adoption (Uniswap v4). The tranche identifier serves as the class key, and the net-asset-value, seniority, and coverage-test state are laid out in namespaced storage keyed by that identifier. This approach gives full control of the tranche model.
- **Against.** More of the semantics are defined by the implementer, which increases the specification surface and reduces reuse of an existing standard.

Current position

Option B (ERC-6909 with ERC-7201) is preferred at this stage, for three reasons. First, the minimal adoption of ERC-3475 — mirrored by the stagnation of EIP-5095 on the cash-flow axis — indicates that the standard is either an imperfect fit or too heavy for the role. Second, the elements of ERC-3475 that would be reused (metadata and net asset value) are the elements most readily defined directly. Third, the value of the proposed standard lies in the engine (components ③ and ④), not in the token; the substrate should therefore be chosen to minimise friction and maximise composability.

Net-new elements, independent of the selection

- An explicit, monotonic seniority index, which neither ERC-3475 nor ERC-6909 provides.
- The waterfall and coverage-test interface; see `../mechanics/oc-ic-coverage-tests.md`.
- Governable, auditable calibration parameters; see `../mechanics/heuristics-vs-plumbing.md`.

This position is provisional and should be revisited if a case for ERC-3475 adoption emerges or a lighter alternative is identified.

Outstanding Questions and Research Direction

The following questions remain open before drafting the normative specification, in approximate priority order. The first two are the most consequential and the most concrete.

1. Loss recognition and net-asset-value oracle (component ②)

A waterfall has no effect until a loss is recognised and quantified, and no existing standard specifies this. Open questions: whether loss is recognised by mark-to-market, accrual, or a default event; who attests to net asset value and what mechanism (for example a dispute or challenge window) constrains that attestation. Many on-chain designs assume an honest net-asset-value feed; the credibility of the standard depends on this component.

2. Implementation review of hardcoded-waterfall protocols (component ③)

A comparison of how existing two-tranche, hardcoded engines compute the split identifies the common structure that the standard should generalise. Targets: Centrifuge version 3, Idle (`IdleCDO.sol`), Maple, Goldfinch, Notional, and Term Finance. For each: the location at which the per-tranche split is computed, the parameters that are configurable versus fixed, and the events emitted.

3. Traditional waterfall mechanics

The precise rules to be encoded: sequential versus pro-rata distribution, turbo and cash-trapping, excess-spread reserves, the formulae for diversion on a coverage breach, the separation of interest and principal waterfalls, and attachment and detachment points. Primary sources: collateralised-loan-obligation and residential-mortgage-backed-security indentures and rating-agency methodologies.

4. Economic security and demand

The principal cause of failure among prior protocols was insufficient demand for the junior tranche. Open questions: which participants hold first-loss capital and at what yield premium; procyclicality and redemption-run dynamics and the gating required to manage them; and the dependence of the senior layer's protection on assumed default correlation, which argues for governable parameters.

5. Failure post-mortems

The failure modes of BarnBridge (regulatory), Saffron, Waterfall DeFi, and Tranche Finance should be examined and translated into requirements for the Security Considerations and Rationale sections (liquidity, junior demand, complexity, and oracle trust).

6. Regulatory considerations (real-world-asset case)

The conditions under which a tranche constitutes a security, informed by the BarnBridge enforcement action, determine whether the compliance branch (ERC-3643, ERC-7518) should be composed in.

7. Specification design

- A minimal interface — distribution events, an epoch or `distribute()` hook, per-tranche `coverageRatio()` and `nav()` views, and trigger-state queries — rather than a complete implementation.
- Composition with ERC-4626 and ERC-7540 beneath, and with the substrate above.
- Conformance vectors for `../test-vectors/`: for example, the senior tranche paid in full before the junior; the junior tranche exhausted before the senior incurs loss; and a coverage breach diverting cash flow.

Pending decisions

- [ ] Token substrate: ERC-3475 versus ERC-6909 with ERC-7201 (currently favouring the latter; see `substrate-fork.md`).
- [ ] Axis scope: the risk axis first, the cash-flow axis subsequently, or a unified treatment.
- [ ] Verification of the token standards used by Untangled, Galaxy, and Tradable; see `../protocols/standards-adoption-matrix.md`.
- [ ] A survey of the Ethereum Magicians forum and ERC repository pull requests for any in-progress tranche or waterfall proposal.